Kara Muhafız — Hafta 2 / Gün 1 Çalışma Raporu

Lokalizasyon: robot_localization Çift-EKF + GPS Füzyonu (2026-10-24)

⚠ BU OTURUM KULLANICI GÖZETİMİ OLMADAN YAPILDI: Kullanıcı C++ mülakat hazırlığı için uzakta olduğundan, bu ve sonraki günlerin kodu Gazebo GUI'de CANLI TEST EDİLMEDEN yazıldı (sandbox kısıtı — Gün 2'de keşfedildi: terminal erişimi gerçek masaüstü oturumundan izole). Kod, standart/belgelenmiş ROS 2 desenleriyle ve mümkün olan her yerde offline doğrulamayla (YAML syntax, parametre yapısı, node'ları Gazebo'suz tek başına başlatıp hata kontrolü) yazıldı. Simülasyonda ilk gerçek testi kullanıcı yapacak.

1. Hedef

Aracın kendi konumunu/yönelimini tahmin edebilmesi (lokalizasyon) — görev şartnamesinin "TF, odometri ve lokalizasyon" kategorisi (15 puan). robot_localization paketinin standart, literatürde en yaygın kullanılan mimarisi seçildi: çift-EKF + navsat_transform.

2. Mimari Kararı ve Gerekçesi

Neden çift-EKF (tek EKF değil)? robot_localization'ın kendi dokümantasyonunda önerilen standart desen budur: yerel EKF (tekerlek odometrisi + IMU) sürekli, sıçramasız bir odom→base_footprint tahmini üretir — bu, kısa vadeli hareket kontrolü (ör. Nav2 local controller) için kritik. Global EKF (+ GPS) ise map→odom TF'ini üretip zamanla biriken sürüklenmeyi (drift) GPS ile düzeltir. GPS tek başına veya doğrudan yerel TF'e karıştırılırsa, GPS'in sıçramalı/gürültülü doğası aracın anlık hareket kontrolünü bozabilir — bu yüzden ayrılıyor.

Önemli değişiklik: controllers.yaml'da ackermann_steering_controller'ın enable_odom_tf'si false yapıldı — artık odom→base_footprint TF'ini EKF yayınlıyor (controller yalnızca ham odometri verisi üretiyor). Bu olmadan iki node aynı TF'i yayınlamaya çalışır, çakışma/kararsızlık oluşurdu.

3. Yapılan Değişiklikler

- config/ekf_local.yaml — yerel EKF (odom çerçevesi): tekerlek odometrisi (x,y,vx,yaw,vyaw) + IMU (yaw,vyaw)
- config/ekf_global.yaml — global EKF (map çerçevesi): aynı ham sensörler + GPS'ten gelen /odometry/gps (x,y)
- config/navsat_transform.yaml — GPS (enlem/boylam) → yerel kartezyen dönüştürücü, wait_for_datum:false (ilk GPS ölçümünü otomatik referans alır)
- launch/localization.launch.py — manual_drive.launch.py'yi dahil edip üç lokalizasyon node'unu da başlatan üst launch dosyası
- controllers.yaml — enable_odom_tf: false (yukarıda açıklandı)
- package.xml — robot_localization bağımlılığı eklendi

4. Offline Doğrulama (Gazebo'suz)

- Tüm YAML dosyaları Python yaml.safe_load ile ayrıştırıldı — syntax hatası yok
- odom0_config / imu0_config / odom1_config dizilerinin uzunluğu 15 olarak doğrulandı (x,y,z,roll,pitch,yaw,vx,vy,vz,vroll,vpitch,vyaw,ax,ay,az sırası — robot_localization'ın beklediği format)
- colcon build --symlink-install başarıyla tamamlandı (3 paket)
- Üç node da (ekf_filter_node_odom, ekf_filter_node_map, navsat_transform_node) Gazebo OLMADAN tek başlarına çalıştırılıp birkaç saniye gözlemlendi — parametre/yapı hatası ÇIKMADI

(robot_localization geçersiz parametre ile anında hata verir, hiçbiri vermedi)
Test EDİLMEYEN kısım: gerçek sensör verisiyle EKF'in ne kadar doğru/kararlı tahmin ürettiği, TF ağacının Gazebo ile birlikte çalışırken tutarlı olup olmadığı. Bunlar simülasyon gerektiriyor.

5. Kod Analizi

5.1 ekf_local.yaml — fuzyon konfigürasyonu mantığı

```
odom0_config: [true,  true,  false,   # x, y, z
               false, false, true,    # roll, pitch, yaw
               true,  false, false,   # vx, vy, vz
               false, false, true,    # vroll, vpitch, vyaw
               false, false, false]   # ax, ay, az
```

Bu 15 elemanlı dizi, robot_localization'a HANGİ ölçümün hangi durumu (state) güncellemesine izin verildiğini söylüyor. Tekerlek odometrisinden yalnızca x,y (konum), yaw (yön), vx (ileri hız), vyaw (dönüş hızı) alınıyor — z/roll/pitch zaten two_d_mode:true ile sıfırlanıyor, vy/vz (yanal/dikey hız) tekerlekli bir araçtan güvenilir ölçülemez. IMU'dan ise yalnızca yaw ve vyaw alınıp ivme (ax,ay,az) BİLEREK dışarıda bırakıldı — ivmeyi çift integre ederek hız/konuma çevirmek hızla sürüklenme (drift) biriktirir; tekerlek odometrisi zaten hız bilgisini doğrudan ve daha güvenilir veriyor.

5.2 localization.launch.py — iki EKF'in çakışmaması

```
ekf_global_node = Node(
    package="robot_localization", executable="ekf_node",
    name="ekf_filter_node_map",
    parameters=[ekf_global_yaml, {"use_sim_time": True}],
    remappings=[("odometry/filtered", "odometry/filtered_map")],
)
```

İki EKF node'u da varsayılan olarak kendi sonucunu "odometry/filtered" adlı topic'e yazmak ister — aynı anda iki node aynı topic'e yazarsa veri karışır. remappings parametresiyle global EKF'in çıktısını "odometry/filtered_map" adına yönlendirdik; yerel EKF varsayılan ismi (odometry/filtered) korudu çünkü navsat_transform_node (ve ileride diğer düğümler) bu ismi varsayılan olarak bekliyor.

6. Bugün Çalışılması Gereken Kavramlar

- Kalman Filtresi (EKF) sezgisel mantığı: birden fazla gürültülü sensörü, her birinin güvenilirliğine (covariance) göre ağırlıklandırıp TEK bir en-iyi-tahmine birleştirmek
- Neden iki ayrı EKF: kısa-vadeli-sürekli (local/odom) vs uzun-vadeli-düzeltmeli (global/map) tahmin arasındaki fark
- GPS'in neden doğrudan değil, navsat_transform üzerinden (enlem/boylam → yerel kartezyen) kullanıldığı
- robot_localization *_config dizilerinin anlamı (15 durum değişkeni, hangi sensörün hangisini güncelleyebileceği)
- ROS 2'de topic remapping (aynı executable'ın birden fazla kopyasını çakışmadan çalıştırma tekniği)

7. Sıradaki Adım

Hafta 2 Gün 2: RViz2 görselleştirme konfigürasyonu, TF ağacının map→odom→base_footprint zincirinin tam dokümantasyonu, ve depthimage_to_laserscan konusunda bir tasarım kararı (zaten çalışan gerçek LiDAR varken gerekli mi, değil mi) belgelenecek.

Bu rapor, Kara Muhafız ROS 2 2025 Parkur Görevi yol haritasının Hafta 2 Gün 1 kapsamını özetler. Kullanıcı dönüşünde ÖNCELİKLE bu lokalizasyon sistemini simülasyonda test etmeli (bkz. Bölüm 4 — test edilmeyen kısım).